1. Comparative Study of Low-Code Development Versus Traditional Application Development Approaches

Aim

To compare low-code development with traditional application development in terms of development speed, coding effort, customisation, cost, maintenance, scalability and suitability for different kinds of software projects.

Theory

Application development covers the design, coding, testing and deployment of software. Two broad approaches are in use today.

Low-code development. A low-code platform offers a visual builder, ready-made components and templates, so most of an application is configured rather than programmed. Code is written only where the logic needs it. Because of this, professional developers deliver faster and trained business users (citizen developers) can also build departmental tools. Typical uses are internal tools, data-entry systems and workflow automation.

Traditional development. The application is hand-coded in languages such as Java, Python, C# or JavaScript on top of chosen frameworks, databases and servers. The team controls every layer of architecture, security and performance, at the price of more time, more skills and more infrastructure work.

Comparison

Parameter	Low-Code Development	Traditional Development
Development time	Short; forms, reports and workflows are configured	Long; every layer is coded and tested
Coding requirement	Minimal, only for special logic	Extensive
Learning curve	Easy	Moderate to difficult
Customisation	Limited to what the platform allows	Highly flexible
Cost	Lower for small and medium projects	Higher at the start
Hosting and security	Provided by the platform	Designed and maintained by the team
Scalability	Moderate, bounded by platform limits	High, design-dependent
Maintenance	Easier; changes are made in the builder	Needs skilled developers and redeployment
Best suited for	Simple to medium business applications	Complex, large enterprise applications

Procedure

1. Study the concepts of low-code and of traditional application development.
2. Review the features of a low-code platform (Zoho Creator): visual forms, reports, pages, workflows and the Deluge scripting language.
3. Study the usual steps of traditional development: requirement analysis, coding, testing, deployment on a server.
4. Compare the two approaches on development time, coding effort, customisation, cost, maintenance and scalability, and record the findings in the comparison table.
5. Relate the comparison to the Smart Lost and Found Management System built in this course: three forms, five list reports, a dashboard and seven form workflows were built on Zoho Creator without writing any server or database code.
6. Draw a conclusion on which approach suits which type of requirement.

Result

The study shows that low-code development greatly reduces development time and coding effort and suits business applications such as the lost and found system. Traditional development needs more time and expertise but offers better flexibility, performance and scalability, so it remains the choice for large and complex systems.